

First Fit Contiguous Memory Allocation

Completed

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the

Select Language : C (GCC 9.2.0)

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int m,n,i,j;
6
7     printf("Enter number of memory blocks:\n");
8     scanf("%d",&m);
9
10    int block[m], original[m];
11
12    printf("Enter size of each block:\n");
13    for(i=0;i<m;i++)
14    {
15        scanf("%d",&block[i]);
16        original[i] = block[i];
17    }
18
19    printf("Enter number of processes:\n");
20    scanf("%d",&n);
21
22    int process[n];
23
24    printf("Enter size of each process:\n");

```

Test Cases

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

> Test Case - 2

First Fit Contiguous Memory Allocation

Completed

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the

Select Language : C (GCC 9.2.0)

```

31
32     for(j=0;j<m;j++)
33     {
34         if(block[j] >= process[i])
35         {
36             int fragment = block[j] - process[i];
37
38             printf("Process %d of size %d is allocated at block %d, starting from index %d\n",
39                    i+1, process[i], j+1, original[j]);
40
41             block[j] = fragment;
42             allocated = 1;
43             break;
44         }
45     }
46
47     if(!allocated)
48     {
49         printf("Process %d of size %d is not allocated\n", i+1, process[i]);
50     }
51 }
52
53 return 0;
54 }
```

Test Cases

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Success ✓

Input :

Expected Output :

```

5
100 500 200 300 600
4
212 417 112 426
```

```

Enter number of
memory blocks:
Enter size of each
block:
Enter number of
processes:
Enter size of each
process:
```